

Homework 1, Machine Learning, Fall 2025

***IMPORTANT* Homework Submission Instructions**

1. All homeworks must be submitted in one PDF file to Gradescope.
2. Please make sure to select the corresponding HW pages on Gradescope for each question.
3. For all coding components, complete the solutions with a Jupyter notebook/Google Colab, and export the notebook (including both code and outputs) into a PDF file. Concatenate the theory solutions PDF file with the coding solutions PDF file into one PDF file which you will submit.
4. You must show work or give an explanation for every question. “Yes” and “No” are not sufficient answers. Always show the steps of your calculations. Only partial credit can be given if you do not explain your answer.
5. Failure to adhere to the above submission format may result in penalties.

As a reminder, don't wait until the last minute to ask for help, because the person you need to speak with might be dealing with many other students close to the deadline. Also, the teaching staff has been instructed that it is not mandatory for them to answer questions on the day the homework is due. So get your questions in early!

You may collaborate on homework, but you must do your own work; we feel this is the best way to learn. If you get stuck, you are welcome to discuss conceptual issues with your classmates or the TAs but you may not copy any code, solutions or proof steps. Do not ask another classmate to perform coding for you. You can look at reference material on the Internet such as ChatGPT, but don't look at that unless you are really stuck, and make sure your answer is complete - don't write “Because I found a theorem that says the answer is X”. You must show your complete work for each question. You are required to state what references you used (if you used an outside source). If you've worked with anyone, write down their name below and make sure they've written your name on their homework as well. **Please copy the following statements at the top of your assignment file:**

Agreement 1: This assignment represents my own work. I have worked with the following people and/or LLMs: _____

Agreement 2: I understand that if I struggle with this assignment that I will reevaluate whether this is the correct class for me to take. I understand that the homework only gets harder.

1 Decision Tree (Theory) (Keshav, Muchang) - 50 pts

The following problem concerns binary classification optimized over n training samples using a 0-1 loss. For simplicity in this question, we focus on decision trees with regularization on the number of leaves. **Please show your work for these questions.**

(a) No regularization

Define our 0-1 loss for tree f on training samples $\mathbf{X}$ with training labels $\mathbf{y}$ as $L(f, \mathbf{X}, \mathbf{y})$:

$$L(f, \mathbf{X}, \mathbf{y}) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}_{f(\mathbf{x}_i) \neq y_i} \quad (1)$$

where $f(\mathbf{x}_i)$ is the prediction of tree f for the i^{th} training sample, and n is the number of training samples.

1. Why would it be a bad idea to optimize Objective 1 as is, without adding sparsity? Give a one sentence answer.
2. For a dataset with k boolean features, what is the deepest valid¹ tree you can make, as a function of k ? You could consider this an implicit constraint on the depth of the tree. (Define the depth of the tree as the number of nodes in the longest path from root to leaf - so a tree with just 1 leaf is depth 1).
3. For Objective 1, how many valid optimal trees are there for the dataset in Table 1? What are these trees (please describe or draw them)? Hint: Trees with different labels in some leaves are not the same, even if their structures are.
4. Among the trees that you identified in part 3, how many unique logical expressions do they represent?

$\mathbf{x}_{.1}$	$\mathbf{x}_{.2}$	$\mathbf{y}$
1	0	0
0	1	1
1	0	0

Table 1: A simple dataset for decision tree optimization

(b) Soft Regularization

Define $s(f)$ as a measure of the sparsity of the tree f (specifically the number of leaves), and adding the regularization penalty $\lambda \in \mathbb{R}_{\geq 0}$, we get the following objective:

$$R(f, \mathbf{X}, \mathbf{y}|\lambda) = L(f, \mathbf{X}, \mathbf{y}) + \lambda s(f) \quad (2)$$

Let $L(f, \mathbf{X}, \mathbf{y})$ remain the 0-1 loss, as in the previous subsection.

1. Suppose an optimal tree for some dataset happens to have only one leaf. What is the largest possible value of $R(f, \mathbf{X}, \mathbf{y}|\lambda)$ for this optimal tree? Note that a tree of depth 1 is considered to have 1 leaf.
2. Consider a tree f_d with depth d that achieves a regularized misclassification error of $R(f, \mathbf{X}, \mathbf{y}|\lambda)$ on some dataset. Now consider a tree f_{d+k} with depth $d+k$ for some $k > 0$, and assume that f_d is a prefix² of f_{d+k} . How much more accurate must f_{d+k} be than f_d for it to achieve a better objective? What is the lowest possible bound for the accuracy improvement of any f_{d+k} in terms of λ and k given that f_d is a prefix of f_{d+k} ?

¹An invalid tree would be one which splits on the same feature multiple times in a single path - so you shouldn't have a tree which splits on feature 1 at the root, then splits on feature 1 again after that. You could still have a tree which splits on feature 1 at the root, then splits on feature 2 in both the left and right children.

²A decision tree f_d is a prefix of a deeper tree f_{d+k} if the structure of f_d corresponds exactly to the top d levels of f_{d+k}

3. Find the smallest depth d (as a function of λ) such that the following claim is true:

For any binary classification dataset $(\mathbf{X}, \mathbf{y})$, any tree f of depth d or greater is guaranteed to be suboptimal with respect to $R(f, \mathbf{X}, \mathbf{y} | \lambda)$.

Define the depth of the tree as the number of nodes in the longest path from root to leaf (so a tree with just 1 leaf is depth 1). Assume $\lambda > 0$.

4. For the dataset in Table 1, how many optimal trees exist for Objective 2 with any $0 < \lambda < \frac{1}{2}$?

2 k-NN, Cross Validation, and ROC (Coding) (Riya) - 30 pts

In this problem, you will implement from scratch the k-NN algorithm on the breast cancer dataset. You will also implement your own cross-validation algorithm in order to tune your model. You may not use a pre-existing k-NN algorithm or cross-validation algorithm such as from Sklearn. If you are unsure whether a package is allowed, feel free to ask in EdDiscussion.

The breast cancer dataset contains 30 feature variables and a target variable which you are trying to predict. The data has already been split into a training and test set for you.

Starter code: https://colab.research.google.com/drive/1xOvn9zzWhMpc5NHk4_7-bmxJzGiHSr7Q?usp=sharing

- (a) Is accuracy or F1 score a more appropriate performance metric to use for this task? Why?
- (b) Implement a k-NN algorithm from scratch to classify the dataset. Use $k = 31$ and the Euclidean distance, and make sure to normalize the data. Report your model's F1 score on the test set.
- Note: See the sklearn article on the MinMaxScaler for more information on how to perform the normalization without leaking information between the train and test sets. Make sure to still implement it from scratch.
- (c) Use cross-validation with 5 folds and the F1 score to tune the value of k and the distance function used (possible distance functions to use could be Euclidean distance, Manhattan distance, or cosine similarity). Make sure to use at least five values of k between 1 and 63, and try at least two distance functions.
- For each distance function, show a plot where the x-axis depicts the value of k and the y-axis depicts the average F1 score for that value of k during cross-validation. Which pair of parameters performed the best?
- Note: You may find the `array_split` method from NumPy helpful when implementing cross-validation.
- (d) Using the best parameters determined in part (c), report the performance of a k-NN classifier on the test set. Compare this to your model in part (b). Is it as you expected?

- (e) We will now directly calculate the ROC curve by sweeping over thresholds.

1. Using the test dataset and the same parameters as part (c), plot the ROC curve of your classifier as points connected by lines. You may implement the ROC curve computation using either of the two methods discussed below.

The ROC curve can be plotted by finding the true positive rate (TPR) and false positive (rate) of the model with different threshold points. For example, if $k = 3$, first find the prediction of each point where the threshold is $k_{pos} > 0, > 1, > 2, > 3$. Then, calculate the TPR and FPR respectively for each threshold and plot them.

A more elegant approach would be grouping points into those with i neighbors that are positive, for i decreasing from k to 0, ordering those groups from $i = k$ positive to $i = 0$ positive votes, and calculating TP and FP cumulatively as i is decreased. So we start with TP and FP over groups with $i = k$, then TP and FP for $i = k - 1$ groups, etc.

2. Calculate the AUC of the ROC curve from scratch.

3 Model Choice (Coding) (Zack) - 20 pts

In this problem, you will explore the *Rashomon Effect*, or the existence of many near-optimal models for a machine learning problem. The *Rashomon set* is the set of all models within ϵ objective of the optimal model. One can define the Rashomon set with respect to either the true risk or the empirical risk – these are called the *true Rashomon set* and the *empirical Rashomon set* respectively. Mathematically, given a dataset $D = \{(x_i, y_i)\}_{i=1}^n$, a hypothesis space $\mathcal{F}$ (e.g. linear models, sparse decision trees, neural networks), and a loss function $L(D, f)$, the empirical Rashomon set is defined as

$$R_{set}(D, \mathcal{F}) := \{f \in \mathcal{F} \mid L(D, f) \leq L(D, f^*) + \epsilon\},$$

where

$$f^* \in \arg \min_{f \in \mathcal{F}} L(D, f).$$

Note that the optimal model may not always be a unique optimizer when the loss function is nonconvex or the hypothesis space is not continuous, so we need the inclusion operator instead of equality.

You will be working with the COMPAS dataset, the hypothesis space of decision trees, and the sparsity-regularized misclassification error loss function,

$$L(D, f) := \frac{1}{n} \sum_{i=1}^n \mathbb{1}[f(x_i) \neq y_i] + \lambda s(f),$$

where $s(f)$ is the number of leaves of f . The COMPAS dataset is a collection of information about arrests in Florida, and the label is whether or not the person in question was arrested within two years of taking a questionnaire at the police department. Among many other use cases (which you will learn more about later in this course), the Rashomon set (or the set of models within ϵ loss of the optimal model) can be used to whittle down a set of models and choose one to deploy. We are going to use the Rashomon set, along with some secondary statistics, to pick a final model to deploy.

Starter code: <https://colab.research.google.com/drive/1JDyGuJOM8bk2laMEBgawqbtfeUVNFqnG?usp=sharing>

Before starting this problem, please download the `compas_hw1_q3.csv` file from Canvas and acquaint yourself with the provided Colab notebook (make a duplicate for yourself to edit). The provided dataset has been preprocessed for you to have a limited set of binary features amenable to computing the Rashomon set of decision trees. There are several provided helper functions in the starter code to help you along.

(a) We will now explore the Rashomon set computed in the problem set.

1. Compute and store the number of leaves of each model in the Rashomon set, the training accuracy of each model, the validation accuracy of each model, and the validation F1 score of each model. You may use the `f1_score` function from scikit-learn.
2. Report the minimum and maximum of each item you computed in part i.

(b) In this part of the problem, you will explore a fairness metric for models called Demographic Parity Difference. Demographic Parity Difference measures the difference in the average prediction of a model across members of certain demographic groups in the dataset. Intuitively, a high Demographic Parity Difference with respect to a sensitive feature means that the model is biased (either correctly or incorrectly) against examples with a particular value of the sensitive feature. Consider a binary feature s , and let

$$I_0 := \{i : x_i[s] = 0\}; \quad I_1 := \{i : x_i[s] = 1\}$$

be the sets of indices where $s = 0$ and $s = 1$, resp. Then the demographic parity difference is defined as the difference between the average prediction when $s = 0$ and the average prediction when $s = 1$:

$$\text{DPD}(D, f, s) := \frac{1}{|I_0|} \sum_{i \in I_0} f(x_i) - \frac{1}{|I_1|} \sum_{i \in I_1} f(x_i).$$

In this subproblem, we will compute the Demographic Parity Difference with respect to the `sex:Female` feature in the COMPAS dataset. For a given model, this will tell us how much more likely the model is to predict males will be arrested than females.

- i. Fill in the provided function skeleton for computing demographic parity difference.
 - ii. Calculate the demographic parity difference of all of the models in the Rashomon set for the `sex:Female` feature. Report the minimum and maximum demographic parity difference inside the Rashomon set.
 - iii. Interpret the minimum and maximum demographic parity differences inside the Rashomon set. What do the results imply about the variety of ways that the models in the Rashomon set decide if someone is likely to be arrested based on their sex?
- (c) We're now in a position to apply secondary statistics and our preferences as data scientists who understand the world beyond numbers to use the Rashomon set to choose a good model to deploy!
- i. Plot histograms, with 20 bins each, of the validation accuracies, demographic parity differences with respect to `sex:Female`, and validation F1 scores in the Rashomon set. We will inspect these histograms to choose cut points to filter the Rashomon set.
 - ii. Let's make sure that we're only looking at models which didn't overfit to the training set. Filter the Rashomon set down to models with validation accuracies > 0.67 . How many models are you left with?
 - iii. Intuitively, we prefer models with smaller demographic parity differences (as long as they're accurate). We want to use someone's sex as sparingly as possible to predict arrest – it may in fact be a useful indicator, but we prefer to make decisions on other bases if at all possible. With this in mind, filter again to preserve only models with validation demographic parity differences < 0.2 . How many models are left?
 - iv. It's bad both to predict someone will be arrested if they don't end up doing so, and also to predict they won't if they would end up being arrested. Let's balance precision and recall in our remaining models. Filter once more to models with a validation F1-score > 0.65 . How many models are you left with?
- (d) Now we're left with a small and manageable subset of remaining models to manually inspect. We're going to look at the logical expressions (Disjunctive Normal Forms, or ORs of ANDs) encoded by the models instead of the trees themselves. Along the way, we'll notice something remarkable about the hypothesis space of decision trees – many trees encode the exact same logical expression!
- i. Use the provided helper code to examine the logical forms of the remaining models. How many unique simplified DNFs did we filter down to?
 - ii. Among the unique simplified DNF forms that you have remaining, use logical equivalence between expressions along with what you know about the features to identify how many of these are truly unique. (Hint: for example, you can't simultaneously have `priors:>3` and `priors:=0`)
 - iii. The few remaining models are very similar to each other! Which one of them would you deploy? There are no wrong answers to this question.